

Deployment Guide

Getting FarmConnect Ghana Live on a \$0/month Stack

A numbered, copy-paste runbook for deploying the web PWA, API, database, and cache to permanently-free tiers. Account creation and dashboard clicks in this guide must be performed by a human (they involve email verification/OAuth) — everything else is a command you can run as-is.

Course	CSCD602 — Advanced Software Engineering, Capstone Project
Group	Angry Nerds
Project title	FarmConnect Ghana
Document version	1.0
Date	12 August 2026
Individual submission by	George Boadu Junior — 22427354

Supporting material for the Project Documentation §Deployment. See `render.yaml` and `vercel.json` at the repo root — this guide explains how to use them.

Target architecture — all free tiers

Vercel Web PWA (static) free forever	Render API — Node/Express free web service	Supabase PostgreSQL 16 free forever, 500MB	Upstash Redis free forever, 10k cmd/day	Google AI Studio Gemini API key free tier
--	---	--	---	---

This combination was chosen specifically because each provider's free tier is *ongoing*, not a time-limited trial — no card required for any of them at this tier, no surprise expiry. The one trade-off: Render's free web service sleeps after ~15 minutes idle and takes ~30–50s to wake on the next request — acceptable for grading/demo use, called out explicitly in the README/User Manual as a known limitation.

EVERYTHING BELOW IS SCRIPTED WHERE A SCRIPT CAN DO IT

You (a human) only need to: (1) sign up for four free accounts, (2) click "New Project/Service" a few times, (3) paste connection strings between dashboards. Every build/deploy step after that is automatic (Vercel/Render both redeploy on every push to `main`).

Step 1 — Push the repo to GitHub

- 1 If not already done: create a GitHub repository and push this project to it. Both Vercel and Render deploy directly from a GitHub repo.

```
git remote -v    # confirm origin points at your GitHub repo
git push origin main
```

Step 2 — Database: Supabase (free Postgres)

- 1 Sign up at supabase.com → **New Project**. Pick any region (closest to your users), set a strong database password, and wait ~2 minutes for provisioning.
- 2 Project → **Settings** → **Database** → **Connection string**. Copy the "**Connection pooling**" (**Transaction mode, port 6543**) URI — this is the one Prisma should use in a serverless-adjacent host like Render's free tier.
- 3 It looks like: `postgresql://postgres.xxx:[PASSWORD]@aws-0-region.pooler.supabase.com:6543/postgres`. Append `?pgbouncer=true` to the end — Prisma needs this flag when talking to Supabase's pooled connection. This full string is your `DATABASE_URL`.

Step 3 — Cache: Upstash (free Redis)

- 1 Sign up at upstash.com → **Create Database** → Redis → pick a region close to your Render region.
- 2 On the database's page, copy the **"Redis URL"** (starts `rediss://` — note the double-s, meaning TLS). This is your `REDIS_URL`. `ioredis` (already used by the API) supports `rediss://` natively — no code change needed.

Step 4 — AI & weather keys

- 1 **Gemini:** aistudio.google.com/apikey → sign in with any Google account → Create API key. No credit card. This is `GEMINI_API_KEY`.
- 2 **OpenWeatherMap:** home.openweathermap.org → sign up → API keys tab → default key. This is `WEATHER_API_KEY`. (Optional — the advisor works without it, just without weather grounding.)
- 3 **GiantSMS** (optional — without it, OTP/SMS just logs to the Render console instead of sending, and the OTP code is additionally echoed back in the API response as `devCode` so the app stays fully usable without a funded gateway): register at giantsms.com, get your API token and a registered sender ID. These are `GIANTSMS_API_TOKEN` / `GIANTSMS_SENDER_ID`.

Step 5 — API: Render

- 1 Sign up at render.com (GitHub sign-in is fastest) → **New** → **Blueprint** → connect your GitHub account → select this repository. Render detects `render.yaml` at the repo root automatically and proposes the `farmconnect-api` service.
- 2 Before clicking "Apply," Render will prompt for every env var marked `sync: false` in `render.yaml`. Fill in: `DATABASE_URL` (Step 2), `REDIS_URL` (Step 3), `GEMINI_API_KEY` / `WEATHER_API_KEY` / `GIANTSMS_*` (Step 4). Leave `CORS_ORIGIN` blank for now — you'll set it in Step 7 once you know your Vercel URL. `JWT_SECRET` / `JWT_REFRESH_SECRET` are auto-generated by Render — you never need to see or set these.
- 3 Click **Apply**. Render installs, builds, runs `prisma migrate deploy` against your Supabase database (creating all 10 tables), and starts the server. Watch the build log for `Already in sync` or a list of applied migrations — both are success.
- 4 Copy your service's URL, shown at the top of its Render dashboard page — e.g. `https://farmconnect-api.onrender.com`. This is your **API base URL**.
- 5 Verify: open `<your-api-url>/health` in a browser. Expect `{"status": "ok", "db": true, "redis": true}` — exactly the response captured in the Testing Report §4.

Step 6 — Web: Vercel

- 1 Sign up at vercel.com (GitHub sign-in) → **Add New** → **Project** → import this repository. Vercel reads `vercel.json` at the repo root automatically.
- 2 Under **Environment Variables**, add `VITE_API_URL` = your Render API URL from Step 5.4 (no trailing slash).
- 3 Click **Deploy**. Vercel runs the build command from `vercel.json` and serves `apps/web/dist`.
- 4 Copy your deployment URL — e.g. `https://farmconnect-ghana.vercel.app`. This is your **live application URL**.

Step 7 — Close the loop: CORS

- 1 Back in Render → your API service → **Environment** → set `CORS_ORIGIN` to your Vercel URL from Step 6.4 (exact origin, no trailing slash — e.g. `https://farmconnect-ghana.vercel.app`).
- 2 Save — Render redeploys automatically. Without this step, the browser will block every API call from the deployed web app with a CORS error.

Step 8 — Seed an administrator account

There is no self-service admin sign-up (SRS §9.3) — run this once, from your own machine, pointed at the production database:

```
cd farmconnect-ghana
DATABASE_URL="<your Supabase connection string>" \
  npx tsx apps/api/prisma/seed.ts --phone 0241234567 --name "Admin Name"
```

That phone number can then log in through the normal phone+OTP screen on the live site and will land in the Admin Portal (`/admin/users`).

Step 9 — Final verification checklist

✓	Check
☐	Live URL loads the Onboarding screen over HTTPS.
☐	Phone+OTP login completes and a Farmer/Buyer account can be created.
☐	A farmer can link Mobile Money and publish a listing.
☐	A buyer can find that listing, place an order, and reach the payment screen.
☐	The seeded admin phone number can log in and see the Admin Portal.
☐	<code><api-url>/health</code> returns <code>db:true, redis:true</code> .

KEEP IT ACCESSIBLE

The coursework requires the live application to remain accessible through grading. Render's free tier and Vercel's free tier do not expire on a timer the way some "trial credit" platforms do — but if either project is deleted, paused, or a billing/verification issue arises on either dashboard, the live URL will go down. Check both dashboards a day before your submission deadline.